Voice Scheduling Agent Project

Oct 5, 2026 · @AI Department

Objective

Build a voice agent that sounds human, schedules meetings by talking to people, puts them on Google Calendar, creates a to-do for the doctor, and keeps reminding the doctor until the task is done. The core of the project is R&D: find and test the best open-source speech-to-text (STT) and text-to-speech (TTS) models for natural conversation. The results feed both Salus (clinical speech) and Janus (the AI front desk).

- Output: a working demo plus a written model comparison with a recommendation
- Stack: Django backend, React frontend, Azure Cosmos DB
- The whole app is voice-enabled: the person talks to it, it talks back, and it can also start the conversation
- Starting point: the department already runs Whisper large-v3 through faster-whisper on the RTX 5090 workstation, so every new STT model is compared against that
- Channel: browser voice first; real phone calls (the department already uses Vonage) come later and are out of scope for this week

What the agent does

A conversation ends with a meeting on the doctor's Google Calendar, a pending to-do, and a reminder that keeps coming until the task is closed.

Step	What happens	Result
1. Schedule	A caller asks by voice for a meeting; the agent confirms doctor, date, time, time zone and purpose	Event created on Google Calendar
2. To-do	The agent creates a task linked to the meeting, with a due time	Task saved as pending
3. Remind	The task is still pending at reminder time; the agent speaks to the doctor in the app and repeats until the task is done or snoozed	Reminder logged
4. Update	The doctor says "done", "snooze an hour" or "move the meeting"	Task and calendar stay in sync

Step	What happens	Result
5. Call the app	The agent starts a voice call in the web app, the doctor answers and talks back	Transcript stored

The agent must say it is an AI at the start of each conversation, and it must ask again when a date, time or name is unclear instead of guessing.

Research questions

Interns answer these in writing, with sources and their own test results, before they commit to a stack.

1. Which open-source STT models are most accurate on spoken scheduling and medical language (names, drugs, dates, times), and which can stream with low delay?
2. Which open-source TTS models sound most natural, and what does it take (latency, GPU, voice choice, pauses, fillers, expressiveness) to make one sound human in a live conversation?
3. Beyond the voice itself, what makes an agent feel human: response delay, turn-taking, interruptions, short acknowledgements, speaking rate?
4. How does a cascade (voice activity detection, STT, LLM, TTS) compare with end-to-end speech models on quality, control and delay?
5. How well do the candidates handle English with Urdu code-switching, which Janus needs?
6. Which licences allow commercial use, and which model terms block it?
7. What hardware and cost per concurrent conversation does each stack need?
8. How can spoken dates and times ("next Tuesday at 3", "after lunch") be turned into exact calendar times with the right time zone?
9. What rules apply to AI voices: disclosure, consent and recording?

Models to start from

The list below is a starting point, not a shortlist: open-source speech moves fast, so interns must survey newer releases and verify every claim themselves.

Layer	Starting points	What to check
Voice activity and turn detection	Silero VAD; semantic turn-detection models such as the LiveKit turn detector	When the caller has really finished speaking, not just paused

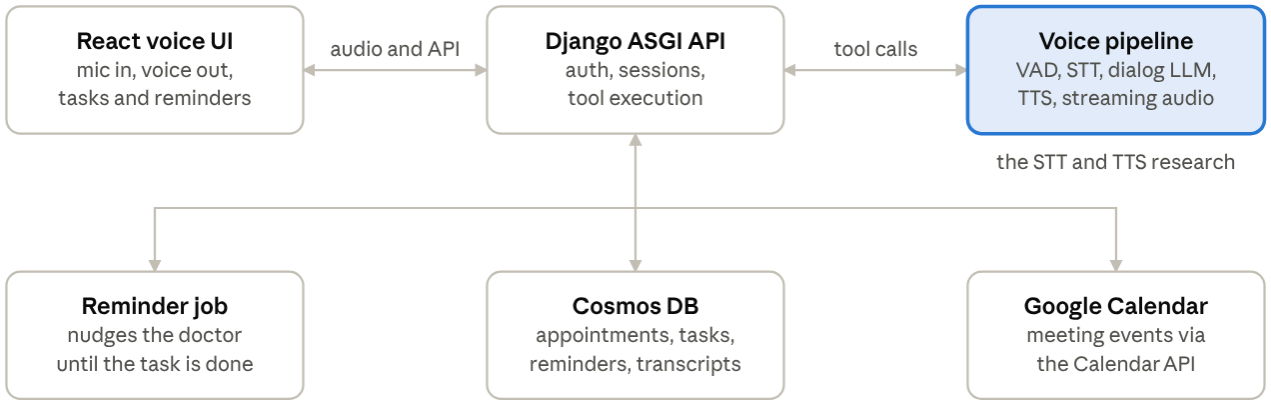
Layer	Starting points	What to check
STT	Whisper large-v3 and faster-whisper (current baseline); distil-whisper; NVIDIA Parakeet; Moonshine; Kyutai STT; Vosk	Streaming support, medical vocabulary, Urdu and English mixing
TTS	Kokoro-82M; Orpheus; Chatterbox; Dia; Sesame CSM; StyleTTS 2; Parler-TTS; CosyVoice	Naturalness, time to first audio, expressiveness, voice options
End-to-end speech	Moshi; Ultravox	Optional comparison against the cascade
Orchestration	Pipecat; LiveKit Agents	Open-source frameworks for real-time voice agents

Licences: some strong models carry non-commercial terms, for example XTTS-v2 and the F5-TTS weights. A licence check for every model in the final comparison is part of the deliverable.

Architecture

The voice pipeline decides what to say and do, Django carries out every action, and Cosmos DB holds the state.

The voice pipeline decides, Django acts, Cosmos DB remembers



architecture · 6 components

- Voice pipeline: voice activity detection, STT, dialog LLM and TTS, streaming at every step; the LLM is swappable and chosen by the interns, open-source preferred

- Tools the agent can call: check availability, create a calendar event, create a to-do, update, snooze or complete a task
- Google Calendar through OAuth 2.0 and the Calendar API, with optional sync of to-dos to Google Tasks
- Cosmos DB (NoSQL API): containers for doctors, appointments, tasks, reminders and call sessions with transcripts; interns choose and defend the partition key. Django's ORM does not support the NoSQL API, so the azure-cosmos Python SDK sits behind a repository layer
- Reminders: a scheduled job finds pending tasks and has the agent speak a reminder in the app; interns propose a fallback for when the app is closed and pick the scheduler (for example Service Bus scheduled messages, Durable Functions timers or Celery beat)
- Transport: browser audio over WebRTC or WebSocket, with both compared; Django runs on ASGI with Channels for WebSockets
- React: microphone button, live captions, a speaking indicator, interrupting by talking, and a task and reminder list

Team split

The project is sized for five interns working in parallel, one role each, sharing one repository and one demo.

Role	Owns	Hands over
STT	Survey, benchmark and streaming setup of speech-to-text models	STT choice and error-rate report
TTS	Survey, listening tests and voice tuning of text-to-speech models	TTS choice and listening-test report
Agent and tools	Voice pipeline, dialog LLM, Calendar and to-do tools	A working agent that books and creates tasks
Voice frontend	React voice UI, live captions, interruption by talking	The voice-enabled app
Backend and data	Django ASGI API, Cosmos DB schema, reminder job, authentication	API, schema and scheduler

Evaluation

Every model and the final stack are judged on numbers the interns measure themselves, on a shared scripted test set.

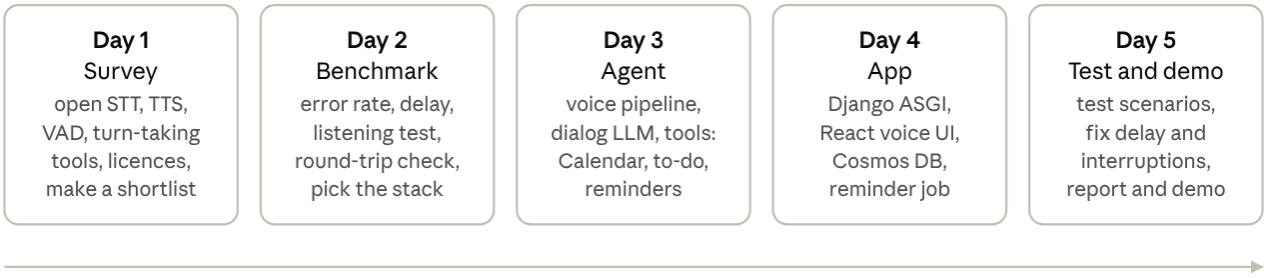
Area	Measure
STT	Word error rate on scheduling and medical-term audio, including English with Urdu; time to first partial result; real-time factor
TTS	Blind listening test scored 1 to 5 by at least 10 listeners; time to first audio; pronunciation of names, drugs, dates and times; round-trip error rate (TTS audio fed back through STT)
Conversation	Delay from end of the caller's speech to the agent's first audio (soft target under 1 second, report what is reached); handling of interruptions; turn-taking errors
Task	Success rate over scripted scenarios: right doctor, date, time and time zone; event on the calendar; task created; reminder fired
Cost	GPU memory, concurrent conversations per GPU, cost per conversation minute

The current Whisper large-v3 setup is the STT baseline, and a hosted engine such as Speechmatics can serve as a paid reference. A small honest result with failure cases is worth more than a polished demo.

One-week timeline

The model choice is made by the end of day 2 so the other three days go to building and testing the agent; the five roles work in parallel from day 3.

Models are chosen by day 2, so days 3 to 5 build and test the agent



one-week plan · 5 days

Deliverables and assessment

The team hands in four things at the end of the week, and is judged mainly on research depth and honest measurement.

1. A model comparison report for STT and TTS, with test results, licences and a recommendation for Salus and Janus
2. A working demo: schedule a meeting by voice, see it on Google Calendar, see the pending to-do, hear the reminder
3. Code for Django, React and the Cosmos DB schema, runnable from a clean checkout
4. A 10 minute demo that includes recordings of what failed

Criterion	What good looks like
Research depth	Surveys current models beyond the starting list and reads their papers, model cards and licences
Voice quality	Blind listening results show the voice sounds natural, not just clear
Conversation	Low delay, clean interruption handling, no talking over the caller
Task reliability	Bookings and tasks are correct across the scripted scenarios
Engineering	Working voice app, clean API, tests, reproducible setup
Communication	Clear report and an honest demo

Data, privacy and voice rules

Only synthetic scenarios and test accounts are used; no real patient, doctor or caller data enters the project.

- No real patient calls, names, numbers or other HIPAA identifiers in audio, transcripts, prompts, logs or Cosmos DB
- Use test Google accounts and test calendars only, never a real doctor's calendar
- Test audio comes from volunteers who agreed to be recorded; no cloning of any real person's voice without their written consent
- The agent says it is an AI at the start of every conversation
- Calls to real phone numbers are out of scope until legal approves; rules on AI voices and consent (for example the US TCPA) apply
- Do not send audio or transcripts to outside services without approval
- Follow the organization's data handling policy and HIPAA obligations at all times